STUDYVERSE

Full-Stack E-Notes Sharing Platform & AWS Cloud Architecture

"Empowering Minds, Sharing Knowledge"

COMPREHENSIVE PROJECT REPORT

Project Title:	StudyVerse - E-Notes Sharing Application
Student Name:	Ayush Kumar
Registration / Roll:	LPU Engineering Student
University:	Lovely Professional University (LPU)
Course:	B.Tech Computer Science & Engineering
Semester:	7th Semester
AWS EC2 Web App:	http://23.20.190.164
AWS RDS MySQL:	dbstudyverse.c6nwkq8cuodu.us-east-1.rds.amazonaws.com
AWS S3 Storage:	studyverse-uploads (us-east-1)
Vercel Preview:	https://e-notes-sharing-application.vercel.app
GitHub Repository:	github.com/Ayush12708/e-notes-sharing-application
Submission Date:	July 2026

Table of Contents

1. Executive Summary & Project Introduction

StudyVerse is a state-of-the-art, full-stack web application developed for university students and academic institutions. It provides a centralized, cloud-hosted platform where students can upload, create, search, bookmark, and discuss academic study notes, lecture slides, code cheat sheets, and exam preparation materials in real time.

Designed specifically to eliminate fragmented file sharing across social media chat groups, StudyVerse features automated moderation workflows, real-time analytics, interactive whiteboard drawing tools, and an enterprise cloud architecture built on AWS EC2, AWS RDS MySQL 8.0, and AWS S3 file storage.

2. Problem Statement & Motivation

In university environments, students encounter significant friction when attempting to access reliable study materials:

- **Fragmented File Distribution** Notes are scattered across temporary WhatsApp groups, personal Google Drive folders, and chat links that expire over time.
- **Lack of Content Quality Verification** Materials uploaded informally often contain errors, missing chapters, or outdated syllabus topics without any peer or admin oversight.
- **No Subject or Semester Categorization** Finding specific subject notes for a given semester or branch requires manual searching through unstructured chat histories.
- **Zero Academic Discussion Context** Static file downloads offer no built-in mechanism for students to ask questions or discuss difficult concepts under a specific note.

3. Project Objectives & Scope

The core engineering goals achieved in StudyVerse include:

- Centralized Repository: Provide a single web destination for all university engineering branches and semesters.
- Rich E-Notes Studio & Whiteboard: Enable students to type formatted digital notes and draw hand-written diagrams using HTML5 Canvas.
- Multi-Format Support: Upload PDF, Word (.docx), PowerPoint (.pptx), Images (.png, .jpg), and ZIP archives.
- Admin Content Moderation: Ensure high quality via a 4-state lifecycle (Draft -> Pending -> Approved / Rejected) with live note preview.
- Enterprise AWS Cloud Hosting: Deploy web services on AWS EC2 (Ubuntu 24.04, Nginx 1.24, Gunicorn), production database on AWS RDS MySQL 8.0, and binary media assets on AWS S3.
- Atomic View Counter: Eliminate race conditions using Django F() expressions during concurrent view and download requests.

4. Cloud Infrastructure Architecture (AWS EC2, RDS, S3, Vercel)

StudyVerse utilizes a multi-tier cloud infrastructure designed for high availability, zero data loss, and seamless scaling:

Layer	AWS / Cloud Service	Configuration Details & Specifications
Virtual Server	AWS EC2 Instance	Ubuntu 24.04 LTS, Public IP 23.20.190.164, Nginx 1.24 Reverse Proxy, Gunicorn 26.0 WSGI Daemon
Database Server	AWS RDS MySQL 8.0	dbstudyverse.c6nwkq8cuodu.us-east-1.rds.amazonaws.com (Port 3306, Schema: notehub_db)
Media File Storage	AWS S3 Bucket	studyverse-uploads (us-east-1), django-storages + boto3 integration for PDF/DOCX/Image uploads
Serverless Host	Vercel Serverless	e-notes-sharing-application.vercel.app (@vercel/python WSGI serverless builder)
Static File Pipeline	WhiteNoise Storage	CompressedStaticFilesStorage serving minified CSS and JS directly via Gunicorn/Nginx
Deployment Script	deploy_ec2.sh	Automated 1-command bash provisioning script installing Python virtualenv, system packages, & services

5. System Design & Django MVT Flow

StudyVerse implements the Model-View-Template (MVT) design pattern:

- **Model Layer (DB ORM)** Encapsulates User Profiles, Notes, Bookmarks, and Comments. Maps directly to AWS RDS MySQL tables.
- **View Layer (Business Logic)** Handles HTTP request dispatching, authentication checks, moderation state transitions, and S3 file streaming.
- **Template Layer (UI Rendering)** Renders responsive HTML5 components styled via a custom 1300+ line CSS design system.
- **Signal Dispatcher (accounts/signals.py)** Uses post_save signals on User model to guarantee Profile row creation regardless of user creation origin.

6. Database Schemas & Relational Data Model (MySQL)

The relational database schema stored in AWS RDS MySQL 8.0 consists of the following primary tables:

Table Name	Primary Fields & Data Types	Relationships & Constraints
auth_user	id (INT PK), username (VARCHAR), email, password, is_staff, is_superuser	Core Django Auth table for multi-user logins
accounts_profile	id (PK), user_id (FK), bio (TEXT), profile_picture, created_at	One-to-One linked to auth_user via Django Signals
notes_note	id (PK), title, subject, branch, semester, file, status, views (INT), user_id (FK)	FK to auth_user; stores status (Pending/Approved/Rejected/Draft) & view counts
notes_bookmark	id (PK), user_id (FK), note_id (FK), created_at	UniqueTogether constraint on (user, note) preventing duplicate bookmarks
notes_comment	id (PK), note_id (FK), user_id (FK), content (TEXT), created_at	Foreign keys to notes_note and auth_user for discussion threads

7. Core Modules & Feature Specifications

- **User Authentication & Profiles** Registration, login, logout, password updates, and single-card smooth UI.
- **Document Upload Studio** Supports uploading PDF, Word, PPTX, Images, and ZIP files directly to AWS S3.
- **Interactive Whiteboard E-Notes** HTML5 Canvas drawing studio with brush sizes, color palette, eraser, and save as draft.
- **Admin Content Moderation Panel** Status filter tabs (Pending, Approved, Rejected, Draft) and live note document preview modal.
- **Student Analytics Dashboard** Aggregates total notes submitted, approved notes, total views, and bookmarks.

8. User Interface & Single-Card Authentication Redesign

To elevate the user experience, the authentication pages were overhauled from split-screen layouts into a sober, centered single-card design:

- **Centered Single-Card Container** Centered `.auth-card-single` layout eliminating visual distraction and side partitions.
- **Academic Color Palette** Ice slate background (`#f8fafc`), deep navy text (`#0f172a`), and subtle royal blue focus rings.
- **Two-Column Grid Fields** Structured form fields for First Name, Last Name, Branch, and Academic Year on registration.

9. Admin Content Moderation & Verification Workflow

StudyVerse enforces quality control through a dedicated moderation dashboard (`/notes/admin-dashboard/`):

- **Filter Tabs** View notes by status: All, Pending, Approved, Rejected, Draft.
- **Document Preview ([View Note])** Staff members can view note text or document contents before taking action.
- **Instant Approval / Rejection** Single-click action buttons update database status and make approved notes immediately visible to all students.

10. URL Routing Structure

URL Pattern	Handler View Function	Access Level & Description
/	home.views.home	Public: Landing page with search & department grid
/accounts/login/	accounts.views.login_view	Public: Single-card sober user login
/accounts/register/	accounts.views.register_view	Public: User registration with signal auto-profile
/dashboard/	dashboard.views.dashboard_view	Student: Personal metrics & uploaded notes list
/notes/browse/	notes.views.browse_notes	Public/Student: Search & filter approved notes
/notes/upload/	notes.views.upload_note	Student: Document file upload to AWS S3
/notes/create-online/	notes.views.create_online_note	Student: Whiteboard canvas & e-notes studio
/notes/admin-dashboard/	notes.views.admin_dashboard	Admin/Staff: Moderation panel with status tabs

11. Teacher Presentation Viva Questions & Detailed Answers

This section provides comprehensive answers to expected viva voce questions for project evaluation:

Q1: What is the overall architecture of StudyVerse?

StudyVerse follows the Django MVT (Model-View-Template) architecture. It is deployed on an AWS EC2 instance running Ubuntu 24.04 with Nginx as a reverse proxy and Gunicorn as the WSGI server. Database storage is handled by AWS RDS MySQL 8.0, and media file uploads are stored in an AWS S3 bucket.

Q2: How does the application handle high concurrent view counts without race conditions?

View counting uses Django F() database expressions: `Note.objects.filter(id=pk).update(views=F("views") + 1)`. This executes the increment directly inside the MySQL database engine atomically, avoiding Python-level read-modify-write race conditions.

Q3: How are User Profiles created automatically when a new User registers or is added via Admin/M

We implemented Django Signals (`post_save`) in `accounts/signals.py`. Whenever a `User` instance is saved, the receiver function automatically instantiates an associated `Profile` record, ensuring 100% data integrity across all creation channels.

Q4: What security measures protect user uploads and data in AWS S3 and RDS?

AWS RDS is restricted via Security Groups on Port 3306. AWS S3 access keys (`boto3`) stream media uploads through signed bucket operations, while Django enforces CSRF tokens on all POST forms and hashed PBKDF2 passwords for user authentication.

Q5: How does the Admin Moderation panel work?

Notes submitted by students enter a `Pending` state. In `/notes/admin-dashboard/`, staff members can preview notes via the `[View Note]` modal, filter notes by status tabs, and click Approve or Reject, which updates the note status and toggles public visibility in the Browse Notes section.

12. Future Scope & Conclusion

StudyVerse successfully bridges the gap in academic resource sharing by offering a reliable, admin-moderated, cloud-hosted platform. With its enterprise architecture on AWS EC2, RDS MySQL, and S3, it demonstrates industry-standard web engineering practices.

Future Roadmap Improvements:

- **AI PDF Summarization** Integrate OpenAI / Gemini API to auto-generate 1-paragraph summary guides for uploaded notes.
- **Mobile Application** Develop React Native mobile app utilizing Django REST Framework endpoints.
- **Elasticsearch Integration** Full-text OCR indexing inside uploaded PDF documents for instant deep keyword search.

Report Generated Successfully for Presentation & Academic Evaluation.